Summary

For quick reference, the following table offers a thorough analysis of the distinctive characteristics, applicability, examples, benefits, and drawbacks of each structural design pattern.

Pattern Name	Distinctive Feature	Applicability	An Example	Pros	Cons
Adapter	Bridges incompatibilities between interfaces	When you want to use an existing class whose interface is not compatible with the rest of the code	Power adapter for different plug types	Enables interoperability of interfaces; reusable	Increases overall complexity of the code
Composite	Represents part-whole hierarchies of objects	When you want to treat individual objects and compositions of objects uniformly	Graphic drawing editor handling shapes and groups of shapes	Simplifies client code; easier management of objects	Design can become overly generalized
Proxy	Provides a placeholder for another object to control access to it	Lazy loading, logging, access control, smart reference	Internet proxy server controlling access to web content	Control object access; reduce cost of expensive operations	Can introduce a level of indirection; may impact performance
Flyweight	Minimizes memory usage by sharing as much data as possible with similar objects	When dealing with large numbers of similar objects with little variation in state	Characters in a word processor	Reduces memory usage; efficient data sharing	Complexity; managing shared state
Bridge	Separates an object's abstraction from its implementation	When you want to avoid a permanent binding between an abstraction and its implementation	Different UI platforms with different backend renderers	Abstraction and implementation can vary independently; scalability	Increased complexity; conceptual overhead
Decorator	Adds responsibilities to objects dynamically	When extending capabilities of a class in runtime	Adding scrolling to a text view dynamically	More flexible than subclassing; avoid feature-loaded classes	Can lead to complex code structure; hard to debug
Facade	Provides a simplified interface to a complex subsystem	When there's a complex system and you want to provide a simple interface to it	Home theater control system	Simplifies interface; reduces dependencies on outside code	Can become a single point of failure; may limit flexibility